CiliaTracks

Instruction Manual

Table of Contents

1 Overview……1
2 Installation……2
2.1 Recommended Installation……2
2.2 Install from GitHub……2
3 Generating raw data from Fiji ImageJ……3
4 Python package usage……4
4.1 Core functions……4
5 Application usage……14
5.1 Quick Summary Model……14
5.2 Manual CiliaTracks Analysis……16
5.3 CNN Model……17
5.4 XGBoost Model……18

1 Overview

CiliaTracks is a comprehensive **Python package** and **dedicated application** that enables clinicians and researchers to analyze cilia-driven particle motility and accurately predict PCD from HVSM data.

The Python package is operating-system independent, while a set of executable applications have been created for Windows, macOS and Linux systems.

1.1 Python Package

The Python package contains the set of visualization, quantitative analysis and prediction computational tools and is easily accessible in the Python framework. The functions are categorized as follows.

- **Trajectory Visualization:** Generate high-quality angular plots that visualize particle trajectory, displacement, and speed.
- **Feature Engineering for Classification:** Automatically extract a comprehensive feature set from track data for input into our XGBoost model for classifying PCD and healthy samples. The feature set could also be used to train and develop new machine learning classifiers.
- **Standardized Image Generation for CNNs:** Produce normalized trajectory images suitable as direct input for our Convolutional Neural Network (CNN) model.
- **Direct PCD Classification:** Make predictions on whether a sample is from a healthy control or a PCD patient using two pre-trained models: an XGBoost model for feature-based classification and a CNN for image-based classification.
- **Automated Fiji ImageJ Script:** The GitHub repository contains a Jython script that automates the entire particle tracking analysis workflow.

1.2 Applications

The corresponding applications provide an accessible and user-friendly alternative for applying the tools/functions in Python. Users can easily upload raw Fiji ImageJ TrackMate data to obtain diagnostic predictions and carry out trajectory analyses.

2 Installation

The applications can be directly downloaded from Zenodo.

The Python package can be installed in two ways. Please note that you must have Python installed to use the package. Python can be downloaded from their official site for your operating system.

2.1 Recommended Installation

The easiest way to install the package is from the Python Package Index (PyPI). This will provide the latest stable release.

```
pip install CiliaTracks
```

2.2 Install from GitHub directly

To get the latest development version, you can install the package directly from the GitHub repository:

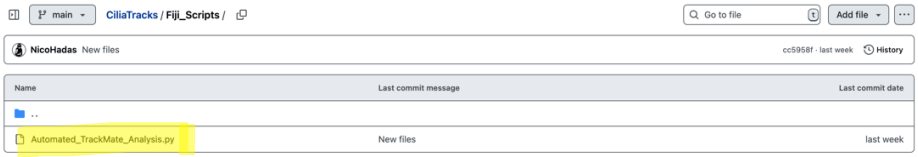
```
git+[https://github.com/NicoHadas/CiliaTracks.git](https://github.com/NicoHadas/CiliaTracks.git)
```

3 Generating raw data from Fiji ImageJ (TrackMate)

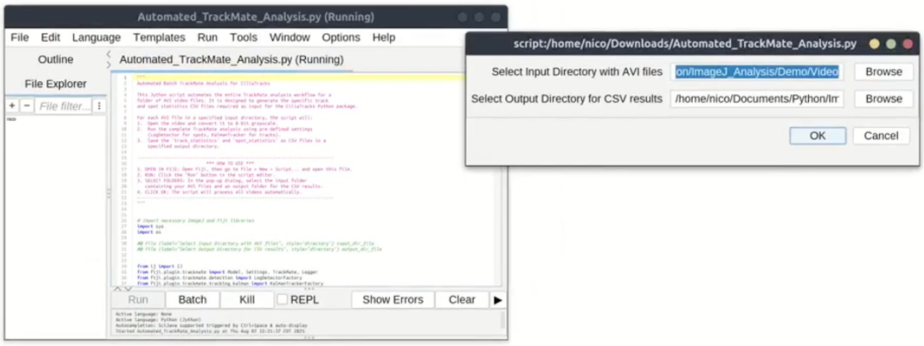
The GitHub repository includes a Jython script that automates the entire TrackMate analysis process. This script will take a folder of raw AVI video files and generate the Tracks.csv and Spots.csv files required for CiliaTracks.

Follow the instructions below to use the script in Fiji ImageJ

1. Download script: Find the script in the Fiji_Scripts folder of this repository.



2. Open the Script Editor in Fiji: In the main Fiji toolbar, navigate to File > New > Script
3. Open the Script File: Select the script file you downloaded
4. Run the Script: Click the "Run" button at the bottom of the editor



5. Select Folders:
   a. Select the input folder containing your .avi video files
   b. Select an output folder where the resulting .csv files will be saved.
6. Click "OK" to start the batch processing. The script will analyze all videos in the input folder and save the results.

Note: If using a confocal microscope, we find it best to modify the script and increase the quality threshold ('THRESHOLD') to 3.5 (line 33) and 'MAX_FRAME_GAP' to 6 (line 112) of Automated_Trackmate_Analysis.py (CiliaTracks/Fiji_Scipts)

4 Python package usage

All functions in this package require two CSV files exported from Fiji/ImageJ using the TrackMate plugin:

- **Tracks.csv**: Contains statistics for each unique track.
- **Spots.csv**: Contains information for every detected spot in the video.

4.1 Core functions

This reference guide provides a detailed description of the core functions available in the CiliaTracks package.

4.1.1 converter()

CiliaTracks.converter(Tracks, Spots, Conversion)

Applies a unit conversion to TrackMate data and returns the converted data as CSV-formatted strings. This is useful for scaling distance-based measurements from pixels to a physical unit like micrometers. This converts feature units in pixels to micrometers, making data more interpretable.

Parameters:

- Tracks: str
    - The file path to the track_statistics.csv file exported from TrackMate.
- Spots: str
    - The file path to the spots_statistics.csv file exported from TrackMate.
- Conversion: float or int (optional)
    - The numerical conversion factor to apply to all distance and speed-related columns in the Tracks and Spots data.

Returns:

- tuple of str
    - A tuple containing two strings: the first is the converted tracks data in CSV format, and the second is the converted spots data in CSV format.

4.1.2 `trajectory()`

`CiliaTracks.trajectory(Tracks, Spots, Conversion=None)`

Generates a polar plot visualizing the complete trajectories of particles from TrackMate data. Each track is normalized to a common origin, and the overall plot is centered around the mean angle of all tracks. The top 150 tracks, sorted by quality, are plotted.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

Returns:

- matplotlib.figure.Figure
  - The figure object for the generated polar trajectory plot, which can be displayed or saved.

4.1.3 `displacement()`

`CiliaTracks.displacement(Tracks, Spots, Conversion=None)`

Generates a polar displacement plot from TrackMate data. Each track is represented as an arrow from the origin, indicating its net movement direction and distance. The top 150 tracks, sorted by quality, are plotted and centered around the mean angle.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

Returns:

- matplotlib.figure.Figure
  - The figure object for the generated polar displacement plot.

4.1.4 speed()

`CiliaTracks.speed(Tracks, Spots, Conversion=None)`

Generates a polar plot of track speed vectors from TrackMate data. Each track is represented by an arrow whose length corresponds to the track's mean straight-line speed. The top 150 tracks are processed and centered around the mean angle.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

Returns:

- matplotlib.figure.Figure
  - The figure object for the generated polar speed vector plot.

4.1.5 `track_ML()`

`CiliaTracks.track_ML(Tracks, Spots, Conversion=None)`

Extracts key track features from TrackMate data and returns them as a pandas DataFrame. These features are designed for use as input to an XGBoost machine learning model for classifying samples.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Track_Number: int, default=150
  - Number of tracks to be considered
- Track_Quality: bool, default=False
  - If True, enables track-quality tier selection (Top/Mid/Bottom) via Track_Quality_Tier
- Track_Quality_Tier: {"Top","Mid","Bottom"}, default="Top"
  - Which quality tier to use when Track_Quality=True:
    - "Top": highest-quality tracks (default behavior)
    - "Good": The next N tracks by quality tracks after top
    - "Mid": middle N tracks by quality rank
    - "Bottom": lowest-quality tracks
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

Returns:

- pandas.DataFrame
  - A single-row DataFrame containing the mean feature values for the top 150 tracks.

4.1.6 `prediction_ML()`

`CiliaTracks.prediction_ML(Tracks, Spots, Conversion=None)`

Extracts features using track_ML and classifies the sample using a pre-trained XGBoost model. The function prints the prediction ('Control' or 'PCD') and the model's confidence score.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

4.1.7 `trajectory_CNN()`

`CiliaTracks.trajectory_CNN(Tracks, Spots, Conversion=None)`

Generates a standardized polar trajectory plot suitable as input for a Convolutional Neural Network (CNN). The plot is minimalistic, with white trajectories on a black background, and all axes and labels removed.

Parameters:

- Tracks: str
  - The file path to the track_statistics.csv file.
- Spots: str
  - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
  - A unit conversion factor to apply to distance values. If None, no conversion is applied.

Returns:

- matplotlib.figure.Figure
  - The figure object for the generated CNN input image.

4.1.8 `prediction_CNN()`

`CiliaTracks.prediction_CNN(Tracks, Spots, Conversion=None)`

Generates a trajectory image in memory and uses a pre-trained Convolutional Neural Network (CNN) to classify the sample. The function prints the prediction ('Control' or 'PCD') and the model's confidence score.

Parameters:

- Tracks: str
    - The file path to the track_statistics.csv file.
- Spots: str
    - The file path to the spots_statistics.csv file.
- Conversion: float, int, or None, *default* None
    - A unit conversion factor to apply to distance values. If None, no conversion is applied.

4.2 Model extraction

Both the pretrained CNN and XGBoost models are available in the GitHub repository under the Models folder. This enables researchers to import the models for their own analyses and provides an option to fine-tune the CNN model.

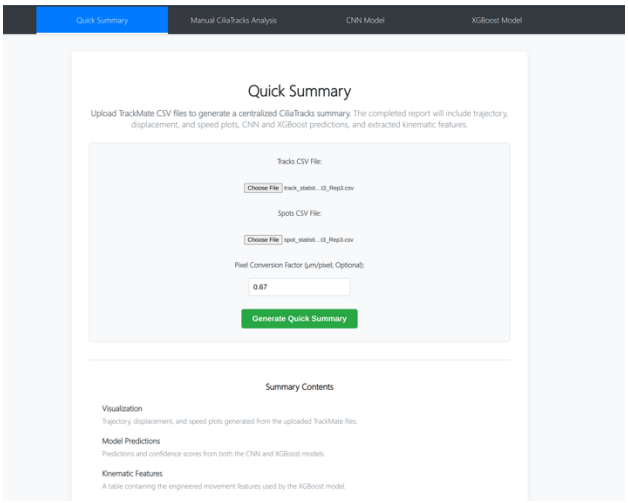
5 Application usage

The corresponding CiliaTracks application has four different tabs:

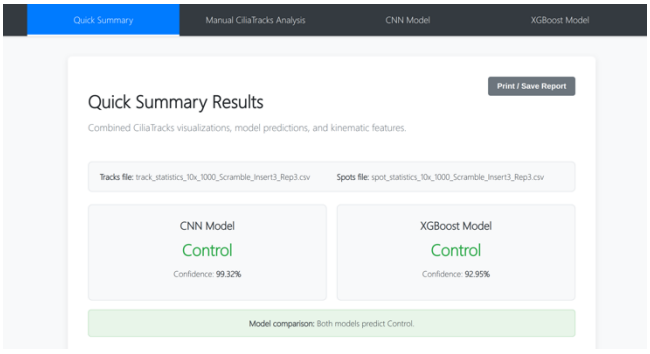
1. Quick Summary: Automated CiliaTracks summary with predictive results from both models, plots and extracted features
2. Manual CiliaTracks Analysis: Non-predictive CiliaTracks functions including plot generation
3. CNN Model: Image-based predictions
4. XGBoost Model: Feature-based predictions

5.1 Quick Summary

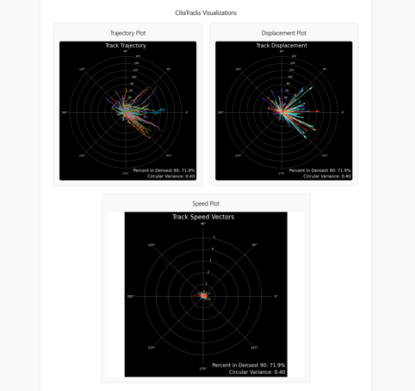
Provides an automated summary of from key CiliaTracks functions. The summary is printable. Input TrackMate raw data CSVs and an optional conversion factor (µm/pixel).



Prediction outcomes from both models are then generated and presented.



Trajectory, displacement and speed plots are also displayed in the summary page.



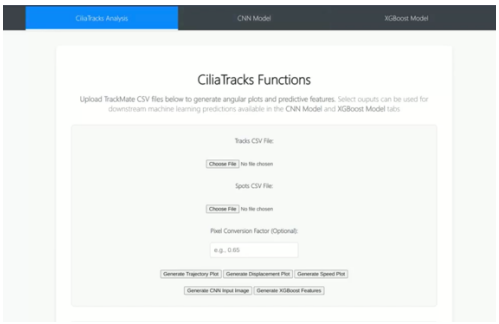
The summary analysis further provides the extracted kinematic features from the recording.

XGBoost Kinematic Features

Feature	Value	Unit
Track displacement	36.1247	µm
Track mean speed	0.4107	µm/frame
Track max speed	3.9908	µm/frame
Track min speed	0.0547	µm/frame
Total distance traveled	60.0823	µm
Max distance traveled	36.9151	µm
Confinement ratio	0.6110	Unitless
Mean straight line speed	0.2321	µm/frame
Linearity of forward progression	0.6147	Unitless
Mean directional change rate	0.8908	radians/frame
Circular variance	0.4040	Unitless
Percent in densest 90°	71.8750	%

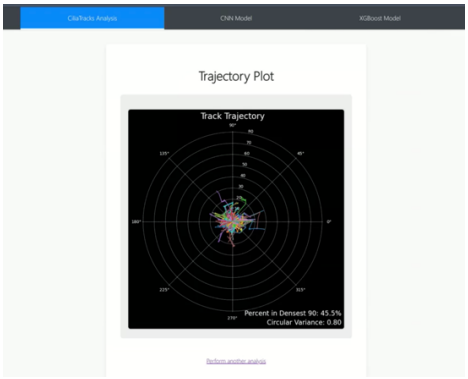
5.2 Manual CiliaTracks Analysis

Five core, non-predictive CiliaTracks functions (from corresponding the Python package) are hosted in the analysis tab. These functions all accept TrackMate raw data CSVs and an optional conversion factor.



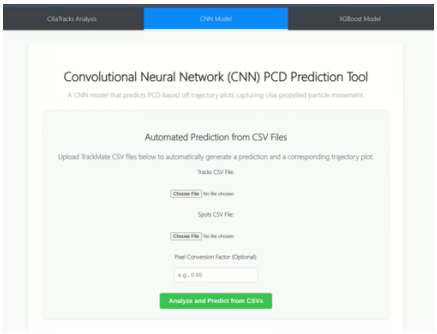
1. Trajectory Plot
2. Displacement Plot
3. Speed Plot
4. CNN Input Image
5. XGBoost Features

For example, a trajectory plot can be easily generated as shown below.

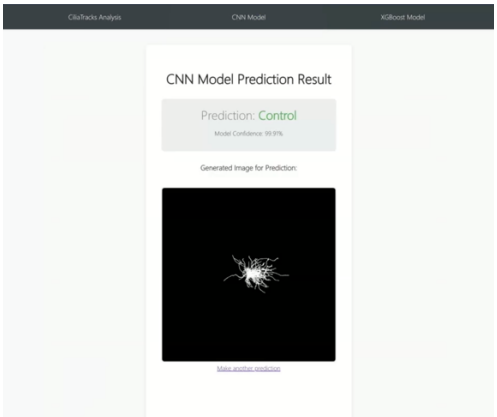


5.3 CNN Model

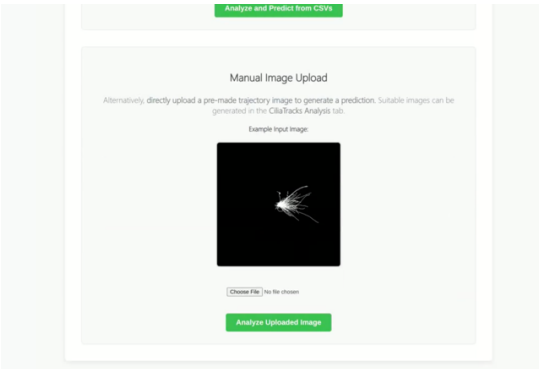
Users can easily generate predictions with TrackMate raw CSVs using our CNN model. Below is the CNN Model tab, where users can input their raw data.



A subsequent prediction is generated along with the processed trajectory plot that served as input for the CNN model.

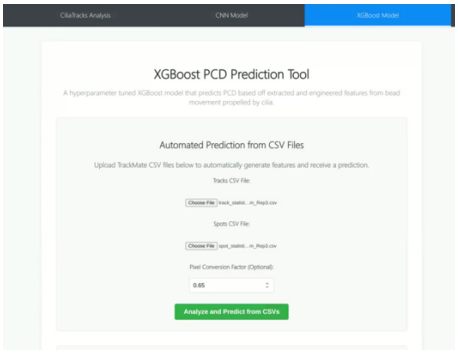


Alternatively, users can also upload processed gray scale trajectory plots directly into the model and generate a prediction.

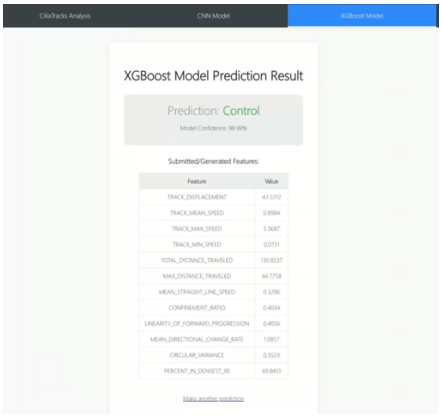


5.4 XGBoost Model

For feature-based predictions, users can upload raw TrackMate CSVs to generate a prediction with the XGBoost model. Below is the XGBoost model tab.



CSV inputs are processed, transformed into a vector of 12 features and then input into the model for a prediction, along with the generated features.



Users also have the option to manually input the 12 predictive track features directly into the model for a prediction.

